# ECE60827: CUDA Assignment 2

Name: Nikhil Reddy Mummadi
GitHub username: mniksr786

**Abstract**
The programs for image noise filtration and maximum pooling were implemented on the CPU and GPU (CUDA) and the performance metrics for different input sizes are discussed.

**Part A: Noise filtration from images**

CUDA Implementation:

In the parallelization of this program, two GPU kernels were defined namely medianFilter_gpu and sortPixels_gpu that utilize shared memory. Each median filter kernel thread operates on an entire filter size where it copies all pixels in the filter window from the global memory to the shared memory, after which the sorting kernel thread sets the median value for the core pixel of the filter window by sorting all the surrounding pixels. All threads within a thread block use the pixel data copied into the shared memory, thus reducing the global memory accesses and improving the code performance.

Results:

The CPU and GPU code was executed for different filter sizes to compare their performance. The CPU runtime scaled up quickly for larger filter sizes, while the GPU runtime increased only to ~1.23 secs for the largest filter size. Table 1 shows the execution time comparison between the CPU and GPU, which proves that the GPU implementation improves latency compared to the CPU code and provides a huge performance boost, especially for larger filter sizes. The filtered image resolution is similar between the CPU and GPU code. However, using larger filter sizes increases smoothing which reduces the local variation causing the pixels to lose the finer details, resulting in a blurred image. Figure 1 illustrates how larger filter sizes cause the filtered images to become blurred.

| Filter Size | CPU Time (secs) | GPU Time (secs) |
|---|---|---|
| 4x4 | 1.14532 | 0.272992 |
| 8x8 | 6.52442 | 0.287867 |
| 16x16 | 30.4417 | 0.537987 |
| 24x24 | 72.3549 | 1.23207 |

**Table 1: Program execution time for various filter sizes**


**Figure 1: Filtered images across multiple filter sizes (labeled in image)**

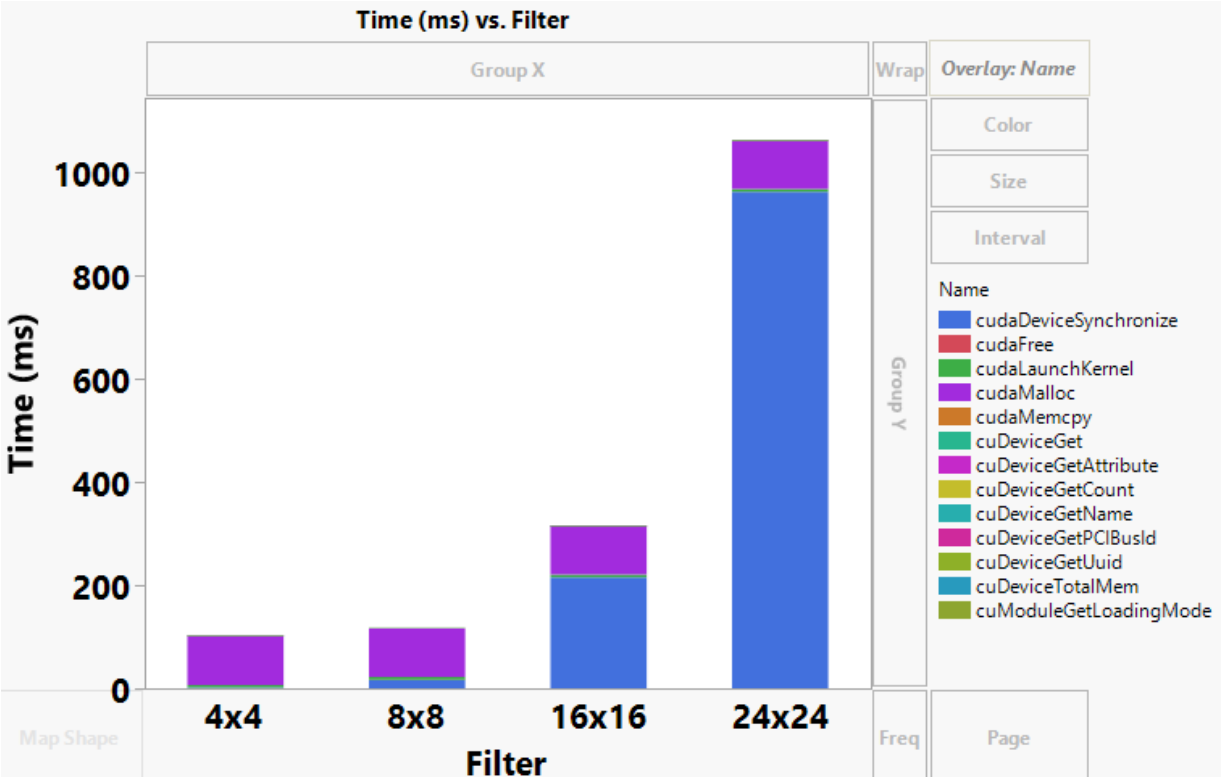


**Figure 2: GPU execution time (ms) breakdown for median-based noise filtering**

As the filter size increases from 8x8 to 24x24, the maximum overhead is from CUDA kernel synchronization to ensure all kernels are complete. But for smaller filter sizes like 4x4, the cudaMalloc API takes most of the execution time. Figure 2 illustrates the detailed time breakdown of all CUDA API calls.

**Part B: Max Pooling**

CUDA Implementation:

In this parallelization strategy, each kernel thread performs the max pooling operation over one pool window of the input tensor to compute the max value element in the output tensor. The code supports multiple tensor sizes and channels, strides for pooling windows, and conditional padding.

Results:

Max pooling for different tensor shapes, and pooling window sizes was executed to compare the CPU and GPU implementations. Figure 3 and Table 2 show the different input cases that were executed along with their execution times. For smaller input tensors and larger pooling windows, the CPU execution is favorable. But for very large input tensors, the GPU parallelization massively improves latency as illustrated for cases 3, 4, and 5. In the current implementation, some overhead is being added due to data transfer between host and device over PCIe which is more prominent for smaller input tensors but can be ignored for very large input tensors given the performance benefit obtained from parallelization.

| Case | Tensor Shape | | Pool Size | | Stride | | Execution time (secs) | |
|---|---|---|---|---|---|---|---|---|
| | Height | Width | Height | Width | Height | Width | CPU | GPU |
| Case_1 | 1000 | 1000 | 5 | 5 | 1 | 1 | 0.582658 | 0.313769 |
| Case_2 | 1000 | 1000 | 10 | 10 | 4 | 4 | 0.142714 | 0.32431 |
| Case_3 | 3000 | 3000 | 10 | 10 | 4 | 4 | 2.55312 | 0.972215 |
| Case_4 | 3000 | 3000 | 20 | 20 | 4 | 4 | 7.88478 | 0.973186 |
| Case_5 | 5000 | 5000 | 20 | 20 | 4 | 4 | 21.9914 | 2.19015 |

**Table 2: Max Pooling execution time comparison between CPU and GPU**

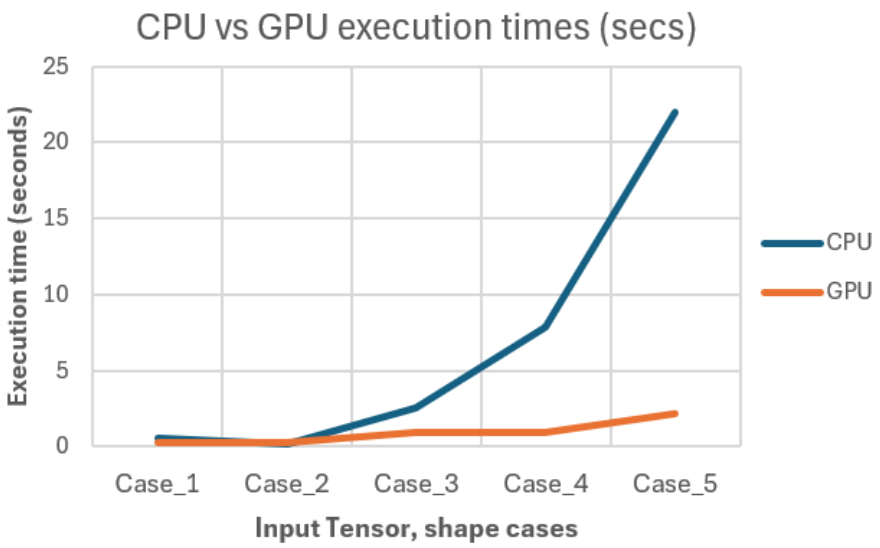


**Figure 3: CPU vs GPU max pooling execution time**

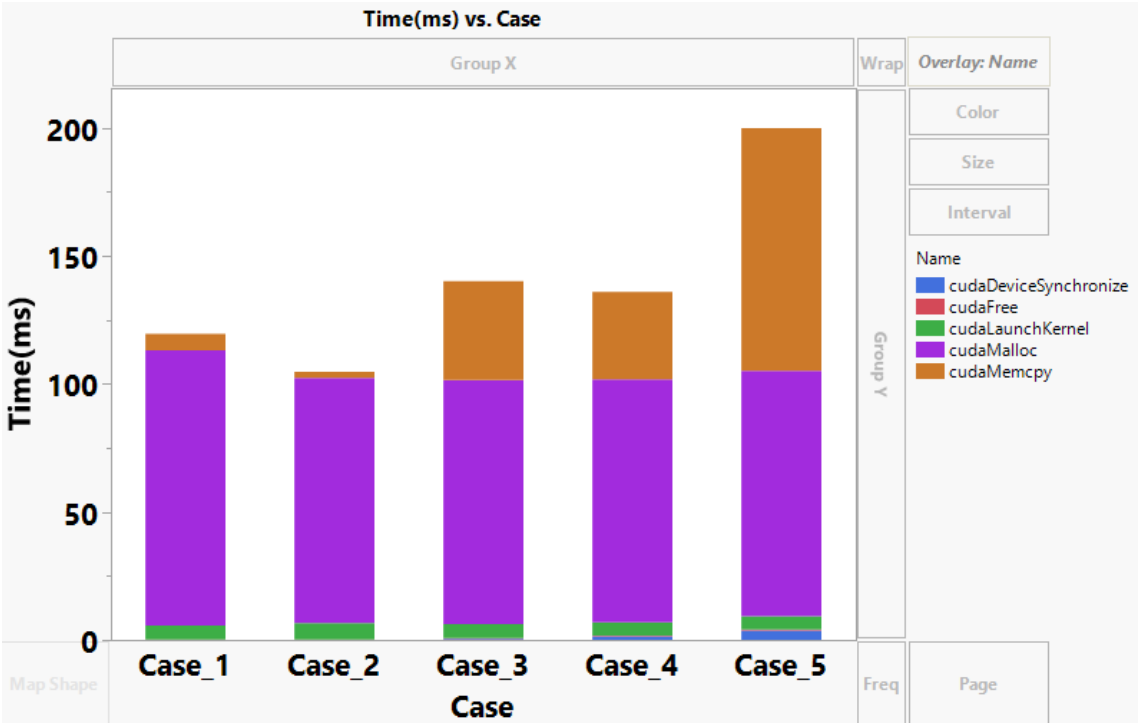


**Figure 4: GPU execution time breakdown for max pooling**

As per Figure 4, the API cudaMalloc takes most of the execution time for all input cases. As the input tensors become large, the APIs cudaMemcpy and cudaDeviceSynchronize also add to the performance overhead as shown in cases 4 and 5.

**Conclusion**

The noise filtration and max pooling algorithms were successfully parallelized on NVIDIA GPUs using CUDA. The programs were profiled across multiple input sizes/cases to examine the performance benefits over CPU and the APIs adding to the performance overhead on GPUs.

*Note: All std::cout console outputs were commented out in the code during profiling of the programs.*